Step 1: Set Up Your Local Environment

Open your terminal (Command Prompt, VS Code terminal, or Cursor) and create a new folder for your app. Run this command to install the required backend packages:

```bash
pip install gradio yt-dlp webvtt-py
```

Step 2: The Application Code (app.py)

Create a file named app.py in your folder and paste this entire block of code into it. This is your entire frontend and backend combined.

```python
import gradio as gr
import yt_dlp
import webvtt
import os
import glob
import shutil

# --- THE BACKEND EXTRACTION ENGINE ---
def extract_channel(channel_url, progress=gr.Progress()):
    # Setup temporary and output directories
    vtt_folder = 'vtt_temp'
    save_path = 'output_megafiles'

    os.makedirs(vtt_folder, exist_ok=True)
    os.makedirs(save_path, exist_ok=True)

    # Clean up previous runs
    for f in glob.glob(f"{vtt_folder}/*"): os.remove(f)
    for f in glob.glob(f"{save_path}/*"): os.remove(f)
```

```
    progress(0.05, desc="Initializing Anti-Bot Bypass...")

    # Configure yt-dlp to mimic a human browser and only pull subs
    ydl_opts = {
        'skip_download': True,
        'writeautomaticsub': True,
        'writesubtitles': True,
        'subtitleslangs': ['en', 'en-US'],
        'subtitlesformat': 'vtt',
        'outtmpl': f'{vtt_folder}/%(id)s.%(ext)s',
        'ignoreerrors': True,
        'quiet': True,
        'sleep_interval': 1,
        'max_sleep_interval': 3,
    }

    progress(0.1, desc="Downloading Subtitles... (This may take 10-20 mins)")

    try:
        with yt_dlp.YoutubeDL(ydl_opts) as ydl:
            ydl.download([channel_url])
    except Exception as e:
        pass # Ignore overall crash if one video fails

    progress(0.6, desc="Download complete. Compiling Mega-Files...")

    # Parse VTTs into NotebookLM-friendly Text Files
    word_limit = 450000
    current_word_count = 0
    file_index = 1
    current_file_content = ""
    saved_files = []

    vtt_files = glob.glob(f"{vtt_folder}/*.vtt")

    if not vtt_files:
        return None, "Error: No subtitles found. Make sure the URL is correct or the channel has captions."

    for filepath in progress.tqdm(vtt_files, desc="Stitching transcripts"):
        try:
            filename = os.path.basename(filepath)
```

```
            vid_id = filename.split('.')[0]
            vtt = webvtt.read(filepath)

            lines = []
            last_text = ""
            for caption in vtt:
                text = caption.text.strip().replace('\n', ' ')
                if text and text != last_text:
                    lines.append(text)
                    last_text = text

            video_text = " ".join(lines)
            if not video_text:
                continue

            formatted_entry = f"\n\n--- SOURCE VIDEO: https://youtu.be/{vid_id} ---\n{video_text}\n"
            current_file_content += formatted_entry
            current_word_count += len(video_text.split())

            if current_word_count >= word_limit:
                save_name = os.path.join(save_path, f"MegaFile_Part{file_index}.txt")
                with open(save_name, "w", encoding="utf-8") as f:
                    f.write(current_file_content)
                saved_files.append(save_name)

                file_index += 1
                current_file_content = ""
                current_word_count = 0

        except Exception:
            continue

    # Save remaining text
    if current_file_content:
        save_name = os.path.join(save_path, f"MegaFile_Part{file_index}.txt")
        with open(save_name, "w", encoding="utf-8") as f:
            f.write(current_file_content)
        saved_files.append(save_name)

    # Clean up temp files
    shutil.rmtree(vtt_folder)

    success_msg = f"✅ MISSION ACCOMPLISHED! Successfully processed {len(vtt_files)} videos
```

```
into {len(saved_files)} Mega-Files ready for NotebookLM."
    return saved_files, success_msg

# --- THE FRONTEND UI ---
with gr.Blocks(theme=gr.themes.Soft(), title="Knowledge Extractor") as app:
    gr.Markdown("# 🧠 Large-Scale Knowledge Extraction Engine")
    gr.Markdown("Enter a YouTube Channel URL (e.g., `https://www.youtube.com/@NicolasCole77`). The system will bypass bot-blocks, download all available subtitles, and compress them into 500k-word Mega-Files for NotebookLM.")

    with gr.Row():
        with gr.Column(scale=2):
            url_input = gr.Textbox(label="Target YouTube Channel URL", placeholder="https://www.youtube.com/@ChannelName")
            extract_btn = gr.Button("Initialize Extraction", variant="primary")

        with gr.Column(scale=1):
            status_output = gr.Textbox(label="System Status", interactive=False)
            file_output = gr.File(label="Download Compiled Mega-Files", file_count="multiple")

    # Connect UI to Backend Logic
    extract_btn.click(
        fn=extract_channel,
        inputs=url_input,
        outputs=[file_output, status_output]
    )

if __name__ == "__main__":
    # Launch the app locally
    app.launch(inbrowser=True)
```

Step 3: Run Your App

In your terminal, execute the script:

```
Bash
python app.py
```

How this Architecture Works

1. **The Web Interface:** As soon as you run that command, it will spin up a local web server (usually at http://127.0.0.1:7860) and automatically open a beautiful GUI in your browser.
2. **Dynamic Progress Tracking:** When you click "Initialize Extraction," the gr.Progress() class in the backend hooks directly into the frontend. You will see real-time progress bars telling you exactly what the app is doing (Bypassing blocks -> Downloading -> Parsing VTTs).
3. **One-Click Downloads:** Once the backend finishes building the files, it automatically packages them and pushes them to the gr.File component on the frontend. You just click the files in your browser to download them to your computer and drag them into NotebookLM.